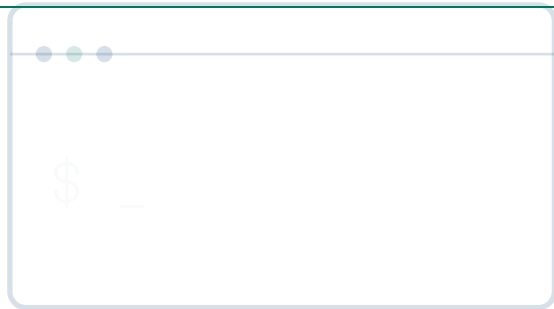


Linux for Computational Chemistry

Lecture 1: Introduction to Linux and the Command Line

Emilio Martínez Núñez

Master in Theoretical and Computational Chemistry



Outline

Why Linux?

The Filesystem

Files and Directories

Getting Help

Permissions

Searching for Files

Process and Job Control

Text Editors: nano and vi/vim

Why Linux?

Why does computational chemistry run on Linux?

- Virtually every HPC cluster and supercomputer runs Linux
- Quantum chemistry and MD packages (Gaussian, ORCA, Gromacs, VASP, [AutoMeKin-X](#)...) are built, tuned, and often only distributed for Linux
- The shell lets you **automate** repetitive tasks: hundreds of single-point calculations, conformer scans, batch post-processing
- Reproducibility: a script is a precise, shareable record of what you did
- Skills transfer directly to any cluster you will ever log into

What is “Linux” exactly?

- **Kernel**: the core program managing hardware, processes, memory
- **Distribution** (Ubuntu, CentOS/Rocky, Debian...): kernel + tools + package manager
- **Shell** (bash, zsh): the program that reads and executes your typed commands
- You almost never touch the kernel directly — you talk to the **shell**

Note

Most HPC clusters use CentOS/Rocky Linux with the bash shell. Everything in this course targets bash.

Anatomy of the prompt

```
[jperez@login1 ~]$
```

- jperez — your username
- login1 — the hostname (the login node of a cluster)
- ~ — current directory (here, your home)
- \$ — ready for input (a # would mean root!)

The Filesystem

The filesystem is a single tree

- Everything starts at the root, / — there is no “C:\”
- Key locations you will meet on a cluster:

<code>/home/you</code>	your personal, backed-up, quota-limited space
<code>/scratch</code> or <code>/work</code>	large, fast, <i>not</i> backed-up space for jobs
<code>/usr</code> , <code>/opt</code>	installed software
<code>/tmp</code>	temporary files, cleared periodically
<code>/etc</code>	system configuration

Tip

Always run large quantum chemistry jobs from `/scratch`, never from `/home` — it is faster and avoids filling your quota.

Absolute vs. relative paths

- **Absolute path:** starts from /, always correct regardless of where you are
- **Relative path:** interpreted from your **current** directory

```
/home/jperez/project/butanol/opt.log # absolute  
project/butanol/opt.log # relative  
. # this directory  
.. # the parent directory  
~ # your home directory
```

Moving around: pwd, ls, cd

```
pwd # print working directory
ls # list contents
ls -l # long format: permissions, size, date
ls -a # include hidden files (dotfiles)
ls -lh # long format, human-readable sizes
ls -lt # sort by modification time
cd project/butanol # move into a subdirectory
cd .. # move up one level
cd # go home
cd - # go to the previous directory
```

Exercise 1.1 — Navigation

Exercise 1.1

Starting from your home directory:

1. Print your current working directory.
2. Create the path `project/butanol/opt1` in one command (hint: look up `mkdir -p`).
3. Move into `project/butanol/opt1` using a relative path.
4. From there, return to your home directory in *two different ways*.
5. List the contents of `project`, including hidden files, sorted by modification time.

Solution

```
pwd
mkdir -p project/butanol/opt1
cd project/butanol/opt1
cd ~ # or: cd
cd $HOME # or: cd - (after having moved once)
ls -lat project
```

Files and Directories

Creating, copying, moving, removing

```
mkdir results # create a directory
rmdir results # remove it, only if empty
touch notes.txt # create an empty file / update timestamp
cp opt.log opt.bak # copy a file
cp -r butanol butanol_bak # copy a directory recursively
mv opt.bak archive/ # move / rename
rm notes.txt # delete a file (no recycle bin!)
rm -r butanol_bak # delete a directory recursively
```

Note

`rm` is permanent. There is no undo. Double-check the target, especially with `-r` and wildcards.

Wildcards

```
*.log # any name ending in .log  
opt?.log # opt1.log, opt2.log... (exactly one character)  
conf[0-9].xyz # conf0.xyz ... conf9.xyz  
conf{1,2,5}.xyz # exactly conf1.xyz, conf2.xyz, conf5.xyz
```

Tip

Before running `rm *.log`, run `ls *.log` first to see exactly what will be deleted.

Exercise 1.2 — File manipulation

Exercise 1.2

1. In `project/butanol/opt1`, create three empty files: `conf1.xyz`, `conf2.xyz`, `conf3.xyz`.
2. Copy all three into a new subdirectory `backup` in a single command using a wildcard.
3. Rename `conf2.xyz` to `conf2_final.xyz`.
4. Remove the `backup` directory and everything in it.

Solution

```
cd project/butanol/opt1
touch conf1.xyz conf2.xyz conf3.xyz
mkdir backup
cp conf*.xyz backup/
mv conf2.xyz conf2_final.xyz
rm -r backup
```

Looking inside files

```
cat opt.log # dump entire file to screen
less opt.log # scroll through a file (q to quit, / to search)
head opt.log # first 10 lines
head -n 20 opt.log # first 20 lines
tail opt.log # last 10 lines
tail -f slurm.out # "follow" a growing file live (Ctrl+C to stop)
wc -l opt.log # count lines
```

Tip

`tail -f` on a running job's output file is the standard way to watch a calculation progress in real time.

Getting Help

Getting help without leaving the terminal

```
man ls # full manual page
ls --help # short usage summary
info coreutils # more detailed manuals (less common)
apropos partition # search manual page descriptions
```

Note

When in doubt about a command's options, `man` is always the first place to check — faster and more reliable than searching online, and it works with no internet on a compute node.

Permissions

Why permissions exist

- Linux is a **multi-user** system — on an HPC cluster, hundreds of people share the same machine
- Every file and directory has an **owner** (a user) and a **group**, plus rules for everyone else
- Permissions decide who can read your inputs, overwrite your results, or run your scripts
- Get them wrong and either: your labmates cannot read shared results, or a script silently refuses to run

The three classes and three actions

Every file/directory defines permissions for **three classes** of people:

owner (u)	the user who created the file
group (g)	members of the file's group
others (o)	everybody else on the system

...and each class can independently have:

r (read)	view contents / list a directory
w (write)	modify contents / add-remove files in a directory
x (execute)	run as a program / <i>enter</i> a directory (cd into it)

Reading 1s -1, character by character

```
-rwxr-xr-- 1 jperez chemgroup 4096 Jun 3 10:21 run_opt.sh
```

1	2-4	5-7	8-10
-	r w x	r - x	r - -
type	owner	group	others

- Character 1: -=regular file, d=directory, l=symbolic link
- Then *three blocks of three*: owner, group, others — always in the fixed order r, w, x
- A - in a slot means that permission is *absent*.
- So here: the owner jperez can read/write/execute; the group chemgroup can read/execute; everyone else can only read

r, w, x mean something different for directories!

	on a file	on a directory
r	view its contents (cat)	list its contents (ls)
w	modify/overwrite it	create/delete/rename entries inside
x	run it as a program	cd into it, access files inside by name

Note

A directory with **r** but no **x** is a common trap: you can list filenames with `ls`, but you cannot `cd` into it or open any file inside it by path. You almost always want **r** and **x** together on directories.

Octal notation: the same thing as numbers

Each permission has a numeric weight, and the three weights for a class simply **add up**:

$$r = 4 \quad w = 2 \quad x = 1$$

digit	sum	symbolic	meaning
7	4+2+1	rwX	read, write, execute
6	4+2	rw-	read, write
5	4+1	r-X	read, execute
4	4	r-	read only
0	-	-	nothing

One digit per class, always in the order **owner-group-others**: `chmod 750` means `rwX`
`r-X --`.

Changing permissions: chmod

```
chmod +x run_opt.sh # symbolic: add execute for everyone
chmod u+x,g-w run_opt.sh # owner +execute, group -write
chmod go-rwx secret.dat # remove ALL access for group and others
chmod 750 run_opt.sh # octal: rwx r-x ---
chmod 644 opt.log # octal: rw- r-- r--
chmod 700 private_notes.sh # octal: only the owner can do anything
```

Tip

Symbolic (u/g/o + +/- + r/w/x) is good for *tweaking* one class. Octal is good for *stating the full picture* in one number — most people default to octal once they are comfortable with the r=4, w=2, x=1 table.

Ownership: `chown` and `chgrp`

```
ls -l opt.log # see current owner:group
chown jperez opt.log # change the owner (often needs sudo)
chgrp chemgroup opt.log # change the group
chown jperez:chemgroup opt.log # change both at once
```

Note

On a shared cluster you will rarely run `chown` yourself — files you create are automatically owned by you and your default group. It is still useful to *recognize* the command when you see it in documentation or scripts.

Exercise 1.3 — Permissions

Exercise 1.3

1. Create a file `run_opt.sh` and inspect its default permissions with `ls -l`. Identify owner, group, and others' permissions separately.
2. Make it executable for yourself only (not group, not others), using octal notation. Predict the 10-character string `ls -l` will show *before* you run it.
3. Verify the change with `ls -l`.
4. Now make it readable and executable by everyone, but writable only by you, using symbolic notation.
5. Create a directory `private` with permissions 700, put a file inside it, then remove *only* the execute permission for yourself (`u-x`) and try `cd private` and `ls private`. Which one fails, and why?

Solution 1.3

Solution

```
touch run_opt.sh
ls -l run_opt.sh # -rw-r--r-- (typical default)
chmod 700 run_opt.sh # predict: -rwx-----
ls -l run_opt.sh
chmod go+rx,go-w run_opt.sh # equivalently: chmod 755 run_opt.sh

mkdir private
touch private/note.txt
chmod u-x private
cd private # fails: Permission denied (no x = cannot enter)
ls private # still works: r lets you list names inside
```

Searching for Files

find: locating files anywhere in a tree

```
find . -name "*.log" # search recursively by exact name pattern
find . -iname "*.LOG" # case-insensitive
find . -maxdepth 2 -name "conf*.xyz" # limit how deep it recurses
find . -type d -name "opt*" # only directories
find . -type f -mtime -1 # regular files modified in the last day
find . -size +100M # files larger than 100 MB
```

Note

Unlike `ls`, `find` searches an entire directory tree at once and can filter by type, age, or size — essential once a project has hundreds of subdirectories.

find with -exec: act on what you find

```
find . -name "*.chk" -exec rm {} \;  
find . -name "*.log" -exec grep -l "Error termination" {} \;  
find . -name "*.log" -mtime +7 -delete
```

- {} is replaced by each matching path; the command ends in \;
- -delete is a shortcut for the common case of removing what matched

Tip

Always run the find command *without* -exec rm or -delete first, to see exactly what would be selected.

Exercise 1.4

Inside your `2-butanol_project` tree (create a few dummy `.log` and `.chk` files to test with):

1. Find every `.log` file anywhere under the project, regardless of depth.
2. Among those, find only the ones that contain the text "Error termination" (combine `find` with `-exec grep -l`).
3. List every file larger than 10 MB anywhere in the tree.
4. Find, but do *not yet delete*, every `.chk` file older than 7 days. Only once you have checked the list, re-run with `-delete`.

Solution

```
find . -name "*.log"
find . -name "*.log" -exec grep -l "Error termination" {} \;
find . -type f -size +10M
find . -name "*.chk" -mtime +7 # inspect first
find . -name "*.chk" -mtime +7 -delete # then delete
```

Process and Job Control

Foreground, background, and interrupting

```
g16 < opt.com > opt.log & # launch in the BACKGROUND (note the &)  
jobs # list background jobs of this shell  
fg # bring the most recent job to the foreground  
fg %1 # bring job number 1 to the foreground  
bg %1 # resume a suspended job in the background
```

- Ctrl+C sends an interrupt — normally terminates the foreground process
- Ctrl+Z *suspends* the foreground process (pauses it, does not kill it)

Inspecting processes: ps and top

```
ps # your processes attached to this terminal
ps aux # every process on the system, full detail
ps aux | grep g16 # is my Gaussian job still running?
top # live, auto-updating view (press q to quit)
```

Note

On an HPC *login* node, ps/top show only what is running there — never long calculations, which belong on compute nodes via the scheduler (Lecture 4).

Stopping a process: kill

```
kill 12345 # ask process 12345 (its PID) to terminate gracefully  
kill -9 12345 # force-kill, when it refuses to stop  
killall g16 # kill by program name instead of PID
```

Tip

Always try a plain `kill` first (it lets the program clean up, e.g. close files properly). Reach for `kill -9` only if the process is genuinely stuck.

Surviving logout: nohup

```
nohup ./run_opt.sh > run.log 2>&1 &
```

Note

Without nohup, a background job launched on a login node is killed the moment you disconnect. In practice, real quantum chemistry jobs are submitted through SLURM (Lecture 4) precisely so you never have to rely on keeping a terminal open — but nohup is essential to know for quick, short tasks on a login node.

Exercise 1.5

1. Launch `sleep 300` (a command that just waits 5 minutes) in the background.
2. Confirm it is running with `jobs` and with `ps`.
3. Bring it to the foreground with `fg`, then suspend it with `Ctrl+Z`.
4. Resume it in the background with `bg`, and confirm with `jobs` that it is running again.
5. Finally, terminate it early with `kill` instead of waiting 5 minutes.

Solution

```
sleep 300 &  
jobs  
ps | grep sleep  
fg # Ctrl+Z to suspend once in the foreground  
bg  
jobs  
kill %1 # or: kill <PID> from ps
```

Text Editors: nano and vi/vim

Surviving with nano

```
nano input.com
```

- Just type to edit — no modes to worry about
- `Ctrl+O` save (“WriteOut”), `Ctrl+X` exit, `Ctrl+K` cut a line
- Simple and safe, but limited — `vi/vim` is worth learning next

Why learn vi/vim?

- `vi` is installed on *every* Unix/Linux system without exception — on a bare cluster node, it may be the only editor available
- `vim` (“Vi IMproved”) is the version found on almost all modern distributions and clusters, and is what we will use here
- It is **modal**: keys do different things depending on which mode you are in — this is the single biggest adjustment for newcomers

The two modes that matter today

Normal mode keys are *commands* (move, delete, save, quit...)

Insert mode keys are typed as *text*, like a normal editor

- vim always opens in **Normal** mode
- Press `i` to enter **Insert** mode and start typing
- Press `Esc` to go back to **Normal** mode — you must do this before any command (including saving or quitting)

Note

The most common beginner mistake: typing `:wq` while still in Insert mode, which just inserts the literal characters `:wq` into the file. Always press `Esc` first.

Opening, saving, quitting

```
vim input.com
```

i	enter Insert mode at the cursor
Esc	return to Normal mode
:w	write (save) the file
:q	quit (fails if there are unsaved changes)
:wq	write and quit
:q!	quit <i>without</i> saving, discarding all changes

Tip

All `:-commands` are typed in Normal mode and confirmed with `Enter`. If you ever feel stuck, press `Esc` then type `:q!` and `Enter` to bail out safely.

Moving around and basic editing in Normal mode

<code>h j k l</code>	left, down, up, right (arrow keys also work)
<code>O / \$</code>	start / end of the current line
<code>gg / G</code>	first / last line of the file
<code>x</code>	delete the character under the cursor
<code>dd</code>	delete (cut) the whole current line
<code>yy</code>	yank (copy) the whole current line
<code>p</code>	paste after the cursor
<code>u</code>	undo (repeat for more)
<code>Ctrl+r</code>	redo

Search and replace in vim

```
/SCF Done # search forward for "SCF Done", Enter to jump to it  
n # jump to the next match  
N # jump to the previous match  
:%s/old/new/g # replace ALL occurrences in the whole file  
:s/old/new/ # replace first occurrence on the current line
```

Note

This is the exact same substitution syntax as sed from Lecture 2 — vim and sed share the same regex heritage, so what you learn in one transfers to the other.

Exercise 1.6 — A first vim session

Exercise 1.6

1. Open a new file `notes.txt` with `vim`. Confirm you start in Normal mode.
2. Enter Insert mode and type three lines of arbitrary text. Return to Normal mode.
3. Move to the first line with `gg`, delete it with `dd`.
4. Undo that deletion with `u`.
5. Search for a word that appears in your text with `/`, then replace every occurrence in the file with `:%s/.../.../g`.
6. Save and quit with `:wq`. Re-open the file with `cat` to confirm your changes were saved.
7. Open it again, make an edit you do *not* want to keep, and exit discarding it.

Solution

```
vim notes.txt
# i, type three lines, Esc
gg
dd
u
/someword (Enter)
:%s/someword/otherword/g
:wq
cat notes.txt
vim notes.txt
# make an edit, then:
:q!
```

- You can navigate, inspect, create, copy, move, and remove files and directories
- You understand absolute vs. relative paths and the standard HPC directory layout
- You can read and set permissions, search a tree with `find`, and control processes and jobs
- You can edit files with both `nano` and `vim`

Homework — Build a project tree (1/2)

Exercise 1.7 (Homework)

Build the following directory structure for a (fictitious) computational study of 2-butanol radical reactions, using a single sequence of shell commands:

```
2-butanol_project/  
|-- inputs/  
|-- outputs/  
|-- scripts/  
'-- README.txt
```

Homework — Build a project tree (2/2)

Exercise 1.7 (Homework), continued

Then:

1. Write one line of text into `README.txt` describing the project (use `vim` or `nano`, or research the `echo "..."` > file redirection we will cover next class).
2. Make everything in `scripts/` eventually executable-ready: set directory permissions to 755.
3. Be ready to show your tree with a single command (hint: look up `tree` or use `ls -R`).